



Warehouse KPI Agent - Developer Guide

Complete guide for developers to understand, setup, and contribute to the Warehouse KPI Agent



Table of Contents

1. Architecture Overview
 2. Technology Stack
 3. Project Structure
 4. Development Setup
 5. Development Workflow
 6. Testing Guide
 7. Contributing
 8. Troubleshooting
-



Architecture Overview

System Architecture

The Warehouse KPI Agent is built as a **stateful AI agent** using:

- LangGraph for orchestration
- Ollama for local LLM inference
- MongoDB for data storage

Core Flow:

User Interfaces → LangGraph State Machine → Tools Layer → Data Layer

Key Architectural Principles

1. Schema-Driven Design

All schemas live in a **single source of truth**: `collections_schema.py`

Benefits:

- No scattered logic
 - Self-documenting
 - Easy updates
 - Type-safe queries
-

2. Keyword-Based Routing

- 80+ domain keywords per collection
 - 5-layer guardrail system ensures accuracy
-

3. Two-Stage LLM Pipeline

Stage 1: Intent Classification

- Extracts intent + entities

Stage 2: Response Formatting

- Converts DB results → human-readable output

4. Stateful Conversations

- SQLite checkpointing
- Thread-based memory
- Supports follow-ups

Technology Stack

Core

- Python 3.12+
- LangGraph
- LangChain

LLM

- Ollama
- qwen2.5:7b

Data

- MongoDB
- PyMongo
- Pandas
- SQLite

APIs & UI

- FastAPI
- Uvicorn
- Streamlit

Export

- openpyxl
- Rich
- Pydantic

Project Structure

Root

```
warehouse_AI/  
├── collections_schema.py  
├── kpi_registry.py  
├── output/  
├── README.md  
├── DEVELOPER_GUIDE.md  
└── warehouse_kpi_agent/
```

Main Package

```
warehouse_kpi_agent/  
├── app/  
├── data_ingestion/  
└── tests/
```

```
├── requirements.txt
├── Makefile
└── .env
```

Key Directories

app/

- config → settings & schema
- db → database logic
- graph → LangGraph workflow
- services → business logic
- tools → collection queries

data_ingestion/

- raw_data → source CSVs
- pruned → cleaned data

tests/

- intent tests
 - KPI tests
 - graph flow tests
-



Development Setup

Steps

1. Clone repo
2. Create virtual env

```
python3 -m venv venv
```

```
source venv/bin/activate
```

3. Install dependencies

```
make install
```

4. Configure .env

5. Start MongoDB

```
make backend
```

6. Setup Ollama

```
ollama pull qwen2.5:7b
```

7. Load data

```
make load-data
```

8. Run tests

```
make test
```

9. Start app

```
make run
```



Development Workflow

Daily Flow

```
make backend
```

```
make run
```

```
make test
```

Adding a New KPI

1. Update `kpi_registry.py`
2. Implement logic in tool
3. Add to KPI list
4. Test

Adding New Collection

1. Add schema
2. Create tool
3. Add keywords
4. Register tool
5. Load data



Testing Guide

Run Tests

`make test`

`python tests/run_tests.py`

Categories

- Intent classification
- KPI calculations
- Graph flow



Contributing

Code Style

- Follow PEP 8
- Use type hints
- Write docstrings

Commit Format

`feat: add KPI`

`fix: correct calculation`

`docs: update guide`



Troubleshooting

MongoDB Issues

`make backend-stop`

`make backend`

Ollama Issues

`ollama list`

`curl http://localhost:11434/api/tags`

Test Failures

make load-data
make test

Import Errors

source venv/bin/activate



Quick Reference

Commands

make setup
make backend
make load-data
make test
make run
make api
make ui

Key Files

Task	Location
Add KPI	kpi_registry.py
Schema	collections_schema.py
Routing	graph/conditions.py
Keywords	intent_classifier.py
Exports	excel_exporter.py

Environment Variables

- MONGODB_URI
 - MONGODB_DB
 - OLLAMA_BASE_URL
 - LLM_MODEL
 - LOG_LEVEL
-



Ready to Start

Run:
make setup
and begin exploring the project!